Otimização Combinatória e Teoria dos Grafos Aplicadas ao Sudoku: Um Estudo da Coloração Própria e Algoritmos de Backtracking Explicitados em Python no Ensino de Matemática

Joao Victor Silva de Sousa

Curso de Licenciatura em Matemática

Disciplina: Metodologia da Pesquisa Educacional III

Instituto Federal do Amazonas

7 de novembro de 2026

Resumo

Este artigo aborda a resolução e a geração automatizada de enigmas do jogo Sudoku sob a ótica da Matemática Aplicada, Otimização Combinatória e Teoria dos Grafos. O tabuleiro 9×9 é formalizado rigorosamente como um grafo simples não orientado $G = (V, E)$ com $|V| = 81$ vértices e $|E| = 810$ arestas, onde a busca por uma solução equivale a encontrar uma 9-coloração própria para o grafo. Desenvolve-se em linguagem Python uma implementação do algoritmo de *backtracking* que opera diretamente sobre o conjunto de adjacências $N(v)$ do grafo explícito, superando a matriz convencional 9×9, e um algoritmo de remoção progressiva com validação de unicidade de solução. Discute-se a relevância pedagógica desta modelagem no âmbito da formação inicial de professores de Matemática, articulando os pilares do Pensamento Computacional e as competências estipuladas pela Base Nacional Comum Curricular (BNCC), sob a normatização técnica da ABNT NBR 14724.

Palavras-chave: Otimização Combinatória; Teoria dos Grafos; Coloração Própria; Backtracking em Grafos; Python; BNCC; ABNT.

---

1 Introdução

No âmbito da Matemática Aplicada e da Otimização Combinatória, a formalização de jogos e quebra-cabeças lógicos constitui um excelente ambiente para o desenvolvimento do raciocínio formal e algorítmico. O Sudoku, um enigma combinatório popular, é frequentemente resolvido na prática por métodos empíricos e heurísticas de eliminação visual. Contudo, sob uma perspectiva matemática rigorosa, a resolução do Sudoku pertence à

classe dos problemas de Satisfabilidade de Restrições (CSP) e pode ser formulada como o problema da k-coloração própria de um grafo.

Na formação inicial de docentes de Matemática, a integração entre conteúdos de Matemática Discreta e ferramentas computacionais atende diretamente às diretrizes da Base Nacional Comum Curricular (BNCC), especialmente no que tange ao desenvolvimento do **Pensamento Computacional** e da **Modelagem Matemática** (habilidades EM13MAT315 e EM13MAT405).

Este trabalho apresenta a modelagem matemática completa do grafo do Sudoku, a implementação explícita do algoritmo de *backtracking* fundamentado no conjunto de vizinhanças $N(v)$ em linguagem Python, e o algoritmo de escavação com garantia de unicidade de solução para geração de tabuleiros, estruturado segundo os ditames da ABNT NBR 14724.

2 Modelagem Matemática: O Grafo do Sudoku

2.1 Definição do Grafo $G = (V, E)$

Definimos o tabuleiro de Sudoku 9×9 como um grafo simples, não orientado e não ponderado $G = (V, E)$, definido pelas seguintes estruturas:

- **Conjunto de Vértices (V):** Cada uma das 81 células corresponde a um vértice $v_{i,j} \in V$, com $i, j \in \{1, 2, \ldots, 9\}$. Assim, $|V| = 81$. Indexando linearmente os vértices para implementação computacional, temos $k = 9(i-1) + (j-1)$, com $k \in \{0, 1, \ldots, 80\}$.
- **Conjunto de Arestas (E):** Uma aresta não orientada $(u, w) \in E$ existe entre dois vértices distintos $u = v_{i,j}$ e $w = v_{k,l}$ se e somente se eles compartilharem ao menos uma relação de conflito.

As relações de conflito são dadas pelas proposições lógicas:

1. **Conflito de Linha:** $i = k$;
2. **Conflito de Coluna:** $j = l$;
3. **Conflito de Subgrade 3×3:** $\lfloor \frac{i-1}{3} \rfloor = \lfloor \frac{k-1}{3} \rfloor \wedge \lfloor \frac{j-1}{3} \rfloor = \lfloor \frac{l-1}{3} \rfloor$.

Portanto, o conjunto de arestas é formalmente expresso por:

$$E = \left\{ (v_{i,j}, v_{k,l}) \in V \times V \;\middle|\; (i,j) \neq (k,l) \;\wedge\; \Big(i = k \;\vee\; j = l \;\vee\; \Big(\Big\lfloor \frac{i-1}{3} \Big\rfloor = \Big\lfloor \frac{k-1}{3} \Big\rfloor \;\wedge\; \Big\lfloor \frac{j-1}{3} \Big\rfloor = \Big\lfloor \frac{l-1}{3} \Big\rfloor \Big) \Big) \right\} \tag{1}$$

2.2 Propriedades Topológicas e Grau dos Vértices

Seja $v \in V$ um vértice genérico. Sua vizinhança aberta $N(v)$ é o conjunto de vértices adjacentes a v:

$$N(v) = \{u \in V \mid (u, v) \in E\} \tag{2}$$

Para calcular o grau $d(v) = |N(v)|$, contabilizam-se as adjacências sem sobreposição:

- 8 vértices na mesma linha;

- 8 vértices na mesma coluna;
- 4 vértices adicionais no mesmo bloco 3×3 que não pertencem à mesma linha nem à mesma coluna.

Assim, G é um grafo 20-regular, ou seja, $d(v) = 20, \forall v \in V$. Pelo Lema dos Apertos de Mão (Handshaking Lemma):

$$|E| = \frac{1}{2} \sum_{v \in V} d(v) = \frac{81 \times 20}{2} = 810 \text{ arestas.} \tag{3}$$

2.3 Formulação como 9-Coloração Própria de Grafos

Uma coloração de G utilizando $k = 9$ cores (correspondentes aos dígitos $\{1, 2, \ldots, 9\}$) é uma função $c : V \to \{1, 2, \ldots, 9\}$. A coloração é dita **própria** se:

$$\forall (u, w) \in E \implies c(u) \neq c(w) \tag{4}$$

Dado um estado inicial com pistas pré-definidas $V_{fixo} \subset V$ e uma atribuição inicial $c_0 : V_{fixo} \to \{1, \ldots, 9\}$, a resolução do Sudoku consiste em estender c_0 para uma coloração própria total $c : V \to \{1, \ldots, 9\}$.

3 Implementação Explícita do Algoritmo de Backtracking em Grafo

A maioria das implementações computacionais tradicionais valida restrições varrendo linhas e colunas através de matrizes bidimensionais. Nesta abordagem de Matemática Aplicada, o algoritmo constrói **explicitamente** a estrutura topológica do grafo $G = (V, E)$ como uma lista de adjacência $N(v)$ e executa o *backtracking* consultando diretamente a vizinhança topológica.

3.1 Cálculo do Conjunto de Cores Viáveis por Operação de Conjuntos

Dado o estado corrente de coloração c, o conjunto de cores indisponíveis para o vértice v é a imagem do seu conjunto vizinho $N(v)$:

$$C_{indisponivel}(v) = \{c(u) \in \{1, \ldots, 9\} \mid u \in N(v) \wedge c(u) \neq 0\} \tag{5}$$

O conjunto de cores viáveis $C_{viavel}(v)$ é obtido estritamente pela diferença de conjuntos:

$$C_{viavel}(v) = \{1, 2, \ldots, 9\} \setminus C_{indisponivel}(v) \tag{6}$$

3.2 Código Python: Grafo Explícito e Backtracking

O trecho em Python a seguir implementa a classe `GrafoSudoku` e a busca por *backtracking*:

```python
class GrafoSudoku:
    def __init__(self):
        # 81 vertices indexados de 0 a 80
        self.vertices = list(range(81))
        # Lista de Adjacencias: N(v) para cada vertice
        self.adj = {v: set() for v in self.vertices}
        self._construir_arestas()

    def _obter_posicao(self, v):
        # Mapeia indice escalar v para (linha, coluna)
        return v // 9, v % 9

    def _construir_arestas(self):
        for u in self.vertices:
            r1, c1 = self._obter_posicao(u)
            b1 = (r1 // 3) * 3 + (c1 // 3)
            for v in self.vertices:
                if u != v:
                    r2, c2 = self._obter_posicao(v)
                    b2 = (r2 // 3) * 3 + (c2 // 3)
                    # Aresta de conflito se compartilham linha, coluna ou bloco 3x3
                    if r1 == r2 or c1 == c2 or b1 == b2:
                        self.adj[u].add(v)

    def obter_cores_validas(self, v, coloracao):
        # Operacao de conjuntos: {1..9} - {c(u) | u em N(v)}
        cores_vizinhos = {coloracao[u] for u in self.adj[v] if coloracao[u] != 0}
        return set(range(1, 10)) - cores_vizinhos

    def resolver_backtracking_grafo(self, coloracao, vertice_idx=0):
        # Condicao de Parada: Todos os 81 vertices coloridos
        if vertice_idx == 81:
            return True

        # Pula vertices com pistas fixas
        if coloracao[vertice_idx] != 0:
            return self.resolver_backtracking_grafo(coloracao, vertice_idx + 1)

        # Selecao de cores validas baseada estritamente em N(v)
        cores_possiveis = self.obter_cores_validas(vertice_idx, coloracao)
        for cor in sorted(cores_possiveis):
            coloracao[vertice_idx] = cor  # Atribui cor
            if self.resolver_backtracking_grafo(coloracao, vertice_idx + 1):
                return True
            coloracao[vertice_idx] = 0  # Backtrack (desfaz atribuicao)

        return False
```

Listing 1: Algoritmo de Backtracking sobre Grafo Explícito em Python

4 Algoritmo de Geração e Escavação com Solução Única

Para gerar um enigma de Sudoku válido e matematicamente bem posto, é necessário assegurar a **unicidade da solução**. O processo se divide em duas etapas:

1. Geração de uma 9-coloração completa válida (tabuleiro resolvido aleatório).
2. Processo de **escavação** (remoção de pistas) com teste de unicidade via poda computacional.

4.1 Código Python: Gerador de Enigmas de Sudoku

```python
import random

class GeradorSudoku:
    def __init__(self):
        self.grafo = GrafoSudoku()

    def contar_solucoes(self, coloracao, vertice_idx=0, limite=2):
        if vertice_idx == 81:
            return 1

        if coloracao[vertice_idx] != 0:
            return self.contar_solucoes(coloracao, vertice_idx + 1, limite)

        total = 0
        cores_validas = self.grafo.obter_cores_validas(vertice_idx, coloracao)
        for cor in cores_validas:
            coloracao[vertice_idx] = cor
            total += self.contar_solucoes(coloracao, vertice_idx + 1, limite)
            coloracao[vertice_idx] = 0
            if total >= limite:  # Poda antecipada se encontrar mais de 1 solucao
                break
        return total

    def gerar_tabuleiro_completo(self):
        coloracao = [0] * 81
        def preencher_aleatorio(v_idx=0):
            if v_idx == 81:
                return True
            cores = list(self.grafo.obter_cores_validas(v_idx, coloracao))
            random.shuffle(cores)
            for cor in cores:
                coloracao[v_idx] = cor
                if preencher_aleatorio(v_idx + 1):
                    return True
                coloracao[v_idx] = 0
            return False

        preencher_aleatorio()
        return coloracao
```

```
40
41      def criar_enigma(self, pistas_desejadas=30):
42          tabuleiro = self.gerar_tabuleiro_completo()
43          indices = list(range(81))
44          random.shuffle(indices)
45
46          removidos = 0
47          alvo_remocao = 81 - pistas_desejadas
48
49          for idx in indices:
50              if removidos >= alvo_remocao:
51                  break
52              cor_removida = tabuleiro[idx]
53              tabuleiro[idx] = 0  # Escava a celula
54
55              # Teste de Unicidade
56              copia = list(tabuleiro)
57              if self.contar_solucoes(copia) != 1:
58                  tabuleiro[idx] = cor_removida  # Reverte se houver mais
    de 1 solucao
59              else:
60                  removidos += 1
61
62          return tabuleiro
```

Listing 2: Gerador de Tabuleiros com Teste de Unicidade em Python

5 Articulação Pedagógica: BNCC e Formação Docente

5.1 O Pensamento Computacional na BNCC

A BNCC (Resolução CNE/CEB nº 1/2022) destaca o Pensamento Computacional como habilidade transversal indispensável na formação matemática contemporânea. A abordagem apresentada mobiliza diretamente os quatro pilares:

1. **Decomposição:** Fracionamento do enigma global em restrições locais de vizinhança $N(v)$.
2. **Abstração (EM13MAT501):** Modelagem do tabuleiro físico em um grafo não orientado $G = (V, E)$.
3. **Reconhecimento de Padrões:** Mapeamento das regras de exclusão mútua como uma k-coloração própria.
4. **Algoritmos (EM13MAT315 e EM13MAT405):** Implementação recursiva em linguagem Python com retrocesso investigativo.

5.2 Normatização ABNT

A estrutura deste artigo atende às prescrições formais da **ABNT NBR 14724** para apresentação de trabalhos acadêmicos e **ABNT NBR 10520** para citações, garantindo o rigor metodológico indispensável à pesquisa em Educação Matemática.

6 Considerações Finais

A modelagem do Sudoku através da Teoria dos Grafos e Otimização Combinatória oferece uma ponte concreta entre o rigor matemático e a Ciência da Computação. O uso do algoritmo de *backtracking* baseado em grafos explícitos em Python permite ao licenciando em Matemática vivenciar a transposição didática de conceitos abstratos, consolidando o Pensamento Computacional e fornecendo subsídios metodológicos para sua atuação docente.

Referências

1. ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724: Informação e documentação — Trabalhos acadêmicos — Apresentação**. Rio de Janeiro: ABNT, 2011.

2. BRASIL. Ministério da Educação. **Base Nacional Comum Curricular (BNCC)**. Brasília: MEC, 2018.

3. BRASIL. Conselho Nacional de Educação. **Resolução CNE/CEB nº 1/2022: Normas sobre Computação na Educação Básica**. Brasília: CNE, 2022.

4. BONDY, J. A.; MURTY, U. S. R. **Graph Theory with Applications**. London: Macmillan, 1976.

5. CORMEN, T. H. et al. **Algoritmos: Teoria e Prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

6. WEST, D. B. **Introduction to Graph Theory**. 2. ed. Upper Saddle River: Prentice Hall, 2001.